

Auditoría Técnica y de Negocio — Tus Suplementos

Fecha: Septiembre 2026 · **Alcance:** Backend (FastAPI/PostgreSQL), Frontend (Next.js), CRONs, Arquitectura, SEO, Seguridad, Modelo de negocio

0. Resumen ejecutivo

El proyecto tiene **una base técnica genuinamente por encima de la media** de un comparador de afiliados: modelo multi-tienda real (Producto 1:N Oferta), motor NLP de clasificación propio, historial de precios, scraping resiliente con caché y backoff, y una capa social/gamificación ambiciosa. Eso es mérito real.

Pero hay un problema estructural: **el proyecto migró de un modelo "un precio por producto" a un modelo "multi-tienda con ofertas" y esa migración se quedó a medias**. Varios scripts de negocio críticos (newsletter semanal, retargeting) siguen apuntando al esquema antiguo y están **fallando en silencio**, **probablemente desde hace tiempo**. Además hay un bug de SEO que muy probablemente está impidiendo que Google indexe tus fichas de producto — que es justo el motor de tráfico que necesitas con 0 visitas.

Antes de invertir en "features nuevas para ser el mayor comparador nacional", hay que apagar estos incendios silenciosos, porque ahora mismo estás construyendo sobre una base que se está autodegradando sin que nadie lo vea.

1. BUGS CRÍTICOS (impacto directo en ingresos/tráfico, no cosmético)

1.1 ● El sitemap probablemente no está indexando ningún producto

`frontend/src/app/sitemap.ts` hace:

```

ts
fetch(`${apiUrl}/api/productos?limit=5000`, ...)
```

Pero el backend define:

```

python
limit: int = Query(100, le=200)
```

Un `limit=5000` **viola la validación de FastAPI** (`le=200`) → la API responde `422 Unprocessable Entity` → `res.ok` es `false` → el sitemap se queda con `dynamicRoutes = []`.

Y aunque esto se arreglara, hay un segundo bug: el código lee `data.items`, pero el endpoint devuelve `{total_resultados, productos}` (ver `schemas.PaginatedProducts`). `data.items` **no existe nunca** → sería

`undefined` de todas formas.

Efecto real: tu `sitemap.xml` casi seguro solo contiene las páginas estáticas (home, about, legal...) y **ninguna de las 1.700+ fichas de producto**. Con 0 tráfico y dependiendo 100% de SEO orgánico, esto es probablemente el bug de mayor impacto de todo el proyecto — Google no puede indexar lo que no encuentra en el sitemap (aunque rastree enlaces internos igualmente, el sitemap acelera y prioriza indexación masiva).

Fix: bajar el límite a bloques de 200 con paginación en el propio `sitemap.ts` (loop de `page=1..N`), y corregir `data.items` → `data.productos`.

1.2 ● Newsletter semanal y retargeting llevan (probablemente) meses fallando en silencio

`backend/newsletter_semanal.py`:

```
python
base_query = db.query(models.Producto).filter(
    models.Producto.precio_anterior != None,
    models.Producto.precio_anterior > models.Producto.precio,
)
```

`backend/retargeting_vistas.py`:

```
python
precio = prod.precio if prod.precio else 0.0
```

El modelo `Producto` en `models.py` **ya no tiene columnas `precio` ni `precio_anterior`** — se migraron a la tabla `Oferta` cuando pasasteis a arquitectura multi-tienda (`Producto` 1:N `Oferta`). Acceder a `models.Producto.precio_anterior` como atributo de clase lanza `AttributeError` en el momento de construir la query.

Ambos scripts tienen el error capturado por un `try/except` genérico, así que **no verás un crash ruidoso** — solo un log `❌ Error crítico...` que nadie revisa en un runner de GitHub Actions que "verde ✅" (el workflow no falla porque el script no propaga la excepción hacia arriba con exit code ≠ 0).

Contraste que lo confirma: `backend/scripts/orquestador.py` y `backend/send_telegram_deals.py` **sí fueron actualizados** correctamente para usar `models.Oferta.precio` — es decir, la migración se hizo a medias, dos scripts se quedaron atrás.

Efecto real:

El workflow `.github/workflows/newsletter.yml` (domingos 08:00 UTC) probablemente **no ha enviado ni un solo email del "Top 5 Chollos"** desde la migración multi-tienda.

El workflow `.github/workflows/retargeting.yml` (diario 17:00 UTC) probablemente **no ha recuperado ni un solo usuario con carrito abandonado / producto visto** desde entonces.

Esto es doble: pierdes reactivación de usuarios logueados Y no tienes forma de saberlo porque el error se traga silenciosamente.

Fix inmediato: cambiar ambos scripts para consultar `[Oferta]` (tal y como ya hace `[orquestador.py]`) y, muy importante, **quitar los `[try/except]` silenciosos de los flujos de negocio críticos** o al menos añadir una alerta (Telegram/email a ti mismo) cuando fallen, para que un fallo de este tipo no vuelva a pasar desapercibido durante meses.

1.3 Tracking de clics roto: `/api/click/{id}` no existe

```
frontend/src/components/TrackedAffiliateLink.tsx llama a:
```

ts

```
fetch(`${apiUrl}/api/click/${productId}`, { method: 'POST' })
```

Ese endpoint **no está definido en** `main.py`. El único mecanismo real de tracking de clics es el contador `clics_count` que se incrementa dentro de `/api/out/{tienda}/{slug}` (el cloaker de afiliados). Además, revisando `producto/[slug]/page.tsx`, este componente **ni siquiera se usa** en la ficha de producto — los enlaces reales son `<a>` planos apuntando a `/api/out/...`.

Efecto real: bajo, porque el tracking real (`clics_count`, que sí alimenta tu orden por "relevancia") funciona correctamente vía `/api/out`. Pero es código muerto que genera un 404 silencioso cada vez que se invoca, y sugiere que hay analítica que crees que existe y no existe. Recomiendo borrarlo o implementarlo de verdad si querías un evento de clic diferenciado del de redirección (por ejemplo, para medir CTR de tienda específica en el modal Quick View).

1.4 ● Catálogo principal 100% client-side rendered (CSR) — mal para SEO

`Catalog.tsx` es `"use client"` y hace el `fetch` de productos dentro de un `useEffect`. Esto significa que **la home (tu página más importante) sirve un HTML inicial sin productos** — Google necesita ejecutar JS y esperar el fetch para "ver" el catálogo. Aunque Googlebot moderno renderiza JS, esto:

Retrasa/complica la indexación de la home como página de listado rico.

Hace que no puedas usar ISR/SSG para servir catálogo pre-renderizado ultra-rápido.

Contradice vuestro propio roadmap en `ARCHITECTURE.md` ("migrar hacia páginas SSR/ISR reales `/proteinas/whye`", `/marcas/hsn`") — es decir, ya sabíais que esto era una debilidad.

Fix recomendado: al menos para las combinaciones de categoría/marca más buscadas, crear rutas dedicadas (`/proteinas`, `/marcas/hsn`, etc.) con `generateStaticParams` + ISR, que hagan el fetch inicial en servidor y pasen los datos como prop inicial al componente cliente (hidratación), en vez de depender 100% de un `useEffect`.

1.5 ● Bug de escala: paginación en memoria, no en base de datos

En `GET /api/productos`, el flujo real es:

python

```
productos_raw_duplicados = query.all() # TRAE TODO lo que cumple filtros, sin LIMIT/OFFSET en SQL
productos = productos_filtrados[offset_real : offset_real + limit] # pagina en Python
```

Con 1.773 productos esto "funciona", pero **no escala**. Cada petición al catálogo:

Trae de la BD todos los productos que cumplen los filtros (con `outerjoin` a `Oferta`), lo que además multiplica filas por cada oferta activa — de ahí el `dedup` manual con `set()` que veo justo después).

Los deduplica y filtra sabores/objetivos en Python.

Solo entonces recorta a la página pedida.

Si tu objetivo es "el mayor comparador nacional" con decenas de miles de productos, este patrón se convertirá en el cuello de botella nº1 de rendimiento y factura de servidor. Es aceptable hoy, es una bomba de relojería a 5.000-10.000+ productos.

Fix recomendado (a medio plazo): mover el filtrado de `sabor`/`objetivo` a la query SQL (usando operadores JSON de PostgreSQL, `@>` o `jsonb_exists_any`), aplicar `LIMIT`/`OFFSET` reales, y resolver el "precio mínimo por producto" con una subquery agregada en vez de traer todas las ofertas a Python.

1.6 ● Doble filtro duplicado (copy-paste) en `main.py`

Dentro de `obtener_productos`, el bloque "4. Filtros Básicos (Formatos, Vegano, Sellos)" y el bloque `solo_ofertas` están **literalmente pegados dos veces** en la función (una vez antes de la búsqueda de texto, otra después). No rompe nada porque aplicar el mismo `.filter()` dos veces es inofensivo en SQLAlchemy, pero es una señal de mantenimiento descuidado: si mañana alguien cambia la lógica de "ofertas reales" en un sitio y no en el otro, tendrás comportamiento inconsistente sin previo aviso. Limpiar y dejar una sola copia.

1.7 ● Endpoints duplicados

`comparar_productos` (`GET /api/productos/comparar`) está **definido dos veces** en `main.py`, idéntico. FastAPI se queda con la última definición (funciona), pero indica descuido en merges de git. Revisar el historial de commits para detectar si hay más duplicaciones escondidas de este tipo.

2. Funcionalidades construidas al 80–90% pero no expuestas (dinero dejado en la mesa)

Esto es interesante: tenéis **infraestructura de producto ya construida en base de datos que nunca llega al usuario**.

Reseñas de sabor (`ResenaSabor`): existe la tabla, existe la relación `producto.resenas`, pero **no hay ni un solo endpoint** (`GET`/`POST /api/resenas`) en `main.py`, ni schema Pydantic para exponerlo, ni componente de UI. Es una función de prueba social (¡justo lo que un comparador necesita para diferenciarse de la ficha fría de la tienda!) que está a medio camino y no visible.

Stacks (rutinas compartibles): puedes crear un stack y añadirle productos (`POST /api/stacks`, `POST /api/stacks/{id}/productos/{id}`), pero **no existe ningún endpoint para listarlos ni verlos** (ni `GET /api/stacks/{id}`), ni están incluidos en `PerfilResponse`, que no tiene campo `stacks`). Es decir: se puede

escribir, pero no se puede leer. La función social "estilo Instagram de suplementos" que mencionáis en la documentación como hito conseguido, en la práctica es inalcanzable desde el frontend/API.

Gamificación (`GymMascota.tsx`): el componente existe y calcula niveles/XP, pero no veo ninguna página que lo monte con datos reales del perfil (`/api/perfil/me`). Verificar si realmente está enlazado en algún sitio o es un componente huérfano.

Recomendación: antes de construir features nuevas, cerrar el círculo de las que ya están al 80%. Cuestan menos terminar que features desde cero y ahora mismo representan trabajo de ingeniería "hundido" sin retorno.

3. Seguridad

Hallazgo	Severidad	Detalle
JWT en <code>localStorage</code>	Media	<code>ProductViewTracker.tsx</code> (y presumiblemente <code>AuthContext</code>) leen el token de <code>localStorage</code> . Es vulnerable a robo vía XSS. Para un sitio con inputs de usuario (bio, username, comentarios de reseña en el futuro) esto es un vector real. Recomendado: cookies <code>httpOnly</code> + <code>SameSite=Strict</code> , o al menos sanitizar agresivamente cualquier input renderizado.
Sin rate limiting	Media-Alta	<code>/api/login</code> , <code>/api/registro</code> , <code>/api/newsletter/subscribe</code> no tienen ningún límite de peticiones. Expuesto a fuerza bruta de credenciales y a spam de suscripciones. Añadir <code>slowapi</code> o un límite a nivel de proxy/Render es trivial y barato.
Sin verificación de email	Baja-Media	El registro no exige confirmar el correo. Cualquiera puede registrarse con emails inventados o ajenos, lo que además ensucia tu base de newsletter/retargeting.
Sin recuperación de contraseña	Media	No hay flujo de "olvidé mi contraseña". Es una fricción de producto real y un futuro ticket de soporte garantizado.
Backdoor de Swagger nombrada explícitamente	Baja	<code>/api/login/swagger</code> — el comentario en el propio código dice " <i>puerta trasera oculta</i> ". Funcionalmente no añade superficie de ataque nueva (misma validación de credenciales que <code>/api/login</code>), pero el naming y el hecho de dejarlo documentado así en el código es mala práctica; al menos protegerlo detrás de una IP allowlist o quitarlo en producción.
Búsqueda con <code>ILIKE '%term%'</code>	Baja (hoy) / Media (futuro)	Los comodines <code>%...%</code> al principio impiden usar índices B-tree estándar. Si el catálogo crece x10, <code>/api/productos/live-search</code> (que se llama en cada tecleo, con debounce de 200ms) puede volverse lento. Ya usáis <code>pg_trgm</code> para el <i>scoring</i> (<code>similarity()</code>), pero conviene extender índices GIN trigram también a las cláusulas <code>ILIKE</code> de filtrado, no solo al ranking.

Nota positiva: el patrón fail-fast en `security.py` y `email_service.py` (`sys.exit(1)` si falta `SECRET_KEY`/`RESEND_API_KEY`) es una buena práctica poco común en proyectos de este tamaño — mantenedlo.

4. SEO — específico, no genérico

Sitemap roto → ver §1.1. Prioridad máxima.

CSR en la home → ver §1.4.

JSON-LD incompleto: en `producto/[slug]/page.tsx` el `AggregateOffer` solo usa `product.price` (la oferta más barata) como `lowPrice` y `highPrice` — son el mismo valor, lo cual no representa realmente un "rango" ni las ofertas de las distintas tiendas. Un `AggregateOffer` con `offers: [...]` individuales por tienda (cada uno con su URL de afiliado) daría a Google más contexto real de "comparador" y podría generar rich snippets con "desde X€ en N tiendas", que es exactamente el tipo de resultado enriquecido que le interesa a un buscador.

Sin canonical por producto: solo hay `alternates.canonical` en el layout raíz (para la home). Cada ficha de producto debería declarar su propio canonical explícito en `generateMetadata`, sobre todo si en el futuro generáis URLs con parámetros de tracking.

`cache: 'no-store'` **en fichas de producto:** cada visita a `/producto/[slug]` hace un fetch fresco al backend, sin ISR. Con 1.700+ productos que cambian de precio unas pocas veces al día (según vuestro propio CRON de 6h), esto es candidato perfecto para `revalidate: 3600` o similar — mejora TTFB, reduce carga al backend y ayuda a Core Web Vitals (que sí es señal de ranking).

Contenido "thin": las descripciones vienen literalmente de los feeds de afiliados (Farma2Go/Sportlive) o scrapeadas de HSN — es decir, texto que **también existe en la tienda original y en Farma2Go/Sportlive tal cual**. Google penaliza contenido duplicado a través de dominios. A medio plazo, generar contenido propio y diferenciado (comparativas editoriales tipo "Mejor creatina 2026", guías de compra, contenido de blog enlazando a fichas) es lo que realmente os dará autoridad de dominio para competir en España frente a agregadores más grandes.

5. Arquitectura y escalabilidad (visión de negocio)

Lo bueno primero, porque es real:

Modelo `Producto` 1:N `Oferta` con historial de precios es **la decisión de arquitectura correcta** para un comparador serio — es lo que os diferencia de un simple "catálogo con enlace de afiliado" y es difícil de replicar rápido por la competencia.

El motor NLP centralizado (`utils.py`) evita duplicación de reglas de negocio entre ingestores — buen instinto de ingeniería.

CRON por bloques horarios para respetar límites de API de TradeDoubler (evitar HTTP 429) muestra madurez operativa.

Lo que preocupa de cara a escalar a "mayor comparador nacional":

Un solo scraper de HSN (`hsn.py`) hace requests secuenciales producto a producto con `time.sleep(0.3-0.8s)` — para un catálogo de HSN que crezca, esto puede tardar horas. Es frágil (depende del DOM de

HSN) y lento. Si HSN cambia su theme (ya migrasteis de uno "Hyvä" antes), esto se rompe sin aviso salvo que reviséis logs manualmente. Vuestro propio roadmap menciona un "CRON Monitor de Scrapers" que avise de roturas — **es una prioridad real, no un nice-to-have**, porque ahora mismo un cambio de HTML en HSN puede dejar el 60-70% de vuestro catálogo desactualizado sin que nadie se entere.

Toda la lógica vive en un backend monolítico FastAPI + scripts sueltos ejecutados vía GitHub Actions.

Es correcto para el volumen actual. Si el catálogo/tráfico crecen x10, vais a necesitar separar ingestión (colas, workers) de la API pública para que un scraping largo no compita por recursos/DB locks con usuarios navegando en producción.

Sin caché de respuesta en `/api/productos` **ni** `/api/config/filtros`. `/api/config/filtros` en particular recalcula categorías/marcas/sabores activos en cada llamada con varios `JOIN` + `GROUP BY` — es un dato que cambia pocas veces al día y se pide en cada carga de la home. Cachearlo (Redis, o incluso in-memory con TTL de 10-15 min) reduciría carga de BD de forma trivial.

6. Modelo de negocio y monetización

Sobre AdSense: vuestra intuición es correcta. Para un comparador de precios, el valor percibido es "objetividad" — meter anuncios de terceros (que a veces son literalmente competidores de las tiendas que comparáis) diluye la confianza justo en el momento de decisión de compra. Yo no lo metería, al menos no display ads genéricos. Alternativas que sí encajan con el modelo:

Afiliación mejor explotada, no más canales de afiliación. Ahora mismo la moneda es el clic a "Ver oferta".

Cosas que aumentan la conversión de ese clic sin dañar UX:

Mostrar reseñas de sabor reales (ya tenéis la tabla, falta exponerla) — la prueba social aumenta conversión más que un banner.

Alertas de precio por email ya construidas (favoritos + retargeting) — **pero rotas** (§1.2). Arreglar esto probablemente tiene más ROI inmediato que cualquier feature nueva, porque ya está pagado en desarrollo.

Un histórico de precios visible en la ficha (el dato ya existe en `historial_precios`, mencionado en vuestro propio roadmap como pendiente de frontend) — es un enorme driver de confianza tipo

"CamelCamelCamel/Keepa" y de urgencia de compra ("mínimo histórico").

Contenido propio (blog/guías) monetizado igual vía afiliación, no vía display ads — atrae tráfico long-tail informacional ("mejor creatina para volumen", "whey vs isolate") que hoy no capturáis nada porque el catálogo es 100% transaccional.

Si en el futuro queréis anuncios, mirad formatos "nativos" de patrocinio (una marca destacada de forma transparente como "Patrocinado", no un banner de terceros) — mantiene control de marca y no compete visualmente con el catálogo.

7. Plan de acción priorizado

Semana 1 — Apagar incendios (0 coste de producto nuevo, solo arreglar lo roto):

Arreglar `sitemap.ts` (paginación real + `data.productos` en vez de `data.items`). — Máximo impacto SEO posible con mínimo esfuerzo.

Migrar `newsletter_semanal.py` y `retargeting_vistas.py` a `Oferta.precio` (copiar el patrón ya correcto de `orquestador.py`).

Añadir alerta (Telegram) cuando cualquier CRON falle, para que esto no vuelva a pasar 6 meses sin detectarse.

Borrar o implementar de verdad `/api/click/{id}` — decidir y limpiar.

Semana 2-4 — Cerrar features a medias: 5. Exponer reseñas de sabor (endpoint + UI mínima en ficha de producto). 6. Exponer stacks (endpoint `GET`) y mostrarlos en el perfil público. 7. Frontend del histórico de precios (ya está el dato, falta un gráfico simple).

Mes 2 — SEO estructural: 8. Migrar la home/catálogo (o al menos las combinaciones top de categoría/marca) a SSR/ISR con rutas propias. 9. Enriquecer JSON-LD con `offers` múltiples por tienda. 10. Empezar contenido editorial propio (2-4 guías/mes) enlazando al catálogo.

Mes 3+ — Escalabilidad: 11. Paginación real en SQL para `/api/productos`. 12. Caché para `/api/config/filtros`. 13. Monitor de scrapers (alerta si un ingestor devuelve 0 productos o cae por debajo de un umbral esperado). 14. Rate limiting básico en login/registro/newsletter.

8. Conclusión honesta

No estáis lejos de tener algo realmente competitivo — la arquitectura de datos (multi-tienda + historial de precios) es más sofisticada que la de muchos comparadores que sí tienen tráfico. El problema no es "nos falta construir más", es que **una parte de lo construido se ha desincronizado silenciosamente y otra parte nunca llegó a verse por el usuario**. Arreglar eso primero os va a dar más ROI en las próximas 4 semanas que cualquier feature nueva, y os deja con una base limpia de verdad para escalar a "mayor comparador nacional" sin arrastrar deuda técnica invisible.